

Pandas y Bases de Datos: manejo de DataFrames

Profesor: Pablo J Piccoli Bornia

Auxiliares: Daniela Longas, M Victoria Tolosa

BASES DE DATOS

Definiciones

- Muchas veces en la industria se confunde BBDD con SGBD
- Una BBDD es un **conjunto interrelacionado de datos**. Puede ser tranquilamente una serie de cuadernos analógicos escritos en lapiz y papel.
- Un SGBD es un gestor de base de datos. Es un software específico para manejar bases de datos digitales.

Comparación

Relacional (SQL)	MongoDB
Tablas	Conjuntos
Registros	Documentos
Atributos/campos	Clave/valor
Las tablas se relacionan entre si	Los conjuntos no están relacionados entre sí
Consulta SQL (universal para todos los SGBD realcionales)	Lenguaje de consulta propio
Estático	Flexible

Comparación

Nombre	Apellido	DNI
Juan	Perez	355556363
Julian	Peña	489993245
Julieta	Fogolin	20999321
Mirna	Cubero	null

```
{  
  "nombre": "Juan";  
  "apellido": "Perez";  
  "DNI": " 355556363"  
},  
{  
  "nombre": "Julian";  
  "apellido": "Peña";  
  "DNI": " 489993245"  
},  
{  
  "nombre": "Julieta";  
  "apellido": "Foglin";  
  "DNI": " 20999321"  
},  
{  
  "nombre": "Mirna";  
  "apellido": "Cubero";  
},
```

Otros tipos de BBDD no relacionales

- Concepto de “Big Table”: Cassandra
- Concepto de “base de datos de grafos”: node4j
- Redis, bases de datos en memoria
- Son todas soluciones puntuales a problemas de la industria en los cuales SQL responde mal

Al final volvemos a excel

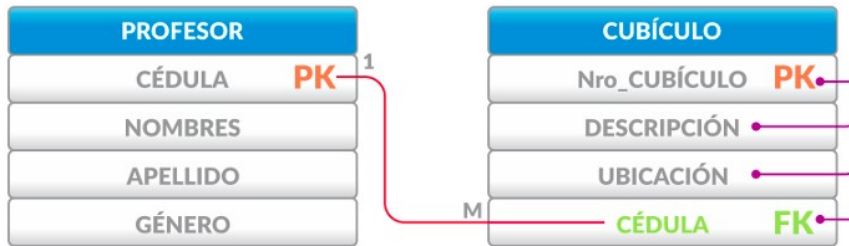
- A pesar de tener varias opciones para almacenar nuestros datos, muchas empresas (especialmente pymes) poseen su información en planillas de cálculo.
- Una serie de datos desparramados en innumerables gsheets, excels y csvs es lo más cercano que tienen a una BBDD
- Por este motivo resulta pertinente aprender **PANDAS**. Aunque antes debemos aprender operaciones de conjunto entre tablas.

JOINS

Conceptos: PK, FK

- En el entorno de una BBDD relacional, una Primary Key (PK) es el mínimo conjunto de atributos que pueden identificar un registro.
- Una FK es un atributo (o conjunto de ellos) que hace referencia a la PK de otra tabla y permite linkear (relacionar) una tabla con la otra

Conceptos: PK, FK



Vemos que la cédula de profesor alcanza para identificar un profesor (entonces decimos que es PK de la tabla “profesor”). El número de cubículo alcanza para identificar a un cubículo, por lo tanto es PK de su tabla. Profesor y Cubículo se relacionan entre sí mediante el campo Cédula, que es PK de Profesor y FK de cubículo

Conceptos: PK, FK

- Vemos que la cédula de profesor alcanza para identificar un profesor (entonces decimos que es PK de la tabla “profesor”).
- El número de cubículo alcanza para identificar a un cubículo, por lo tanto es PK de su tabla.
- Profesor y Cubículo se relacionan entre sí mediante el campo Cédula, que es PK de Profesor y FK de cubículo

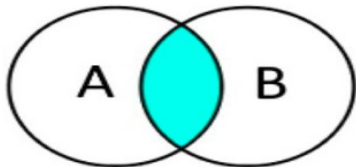
Conceptos: Cardinalidad

La cardinalidad muestra con cuantos registros de una tabla Y se relaciona un registro de la tabla X. Estas pueden ser:

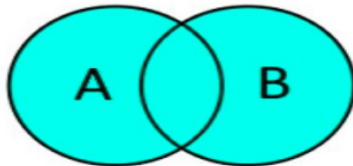
- 1:1: Un registro de la tabla X se relaciona con un solo registro de la tabla Y. Una tabla persona y otra historial médico. Dada una persona hay un historial médico posible y viceversa.
- 1: N : Un registro de la tabla X se relaciona con muchos de la tabla Y, pero no al revés. Tabla club de futbol y tabla hinchas: un hincha puede tener solo un club, un club puede tener varios hinchas.
- N:N : Muchos a muchos, un registro de la tabla X se relaciona con varios de la tabla Y y al revés también. Tabla vuelos y tabla turista. Un turista puede tener varios vuelos, un vuelo suele tener varios turistas

Conceptos: Operaciones de conjuntos

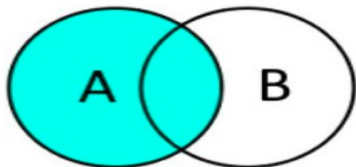
INNER JOIN



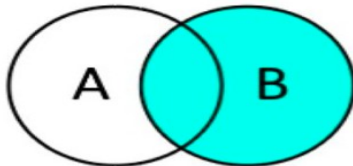
FULL JOIN



LEFT JOIN



RIGHT JOIN



Inner Join

DNI	Nombre	id_club
41555676	Juan Perez	1
38999432	Damian Lopez	4
33455654	Julio Suarez	1

Nombre_club	año_fundacion	id
River Plate	1901	1
Boca Juniors	1905	2

DNI	Nombre	id_club	Nombre_club	año_fundacion
41555676	Juan Perez	1	River Plate	1901
33455654	Julio Suarez	1	River Plate	1901

Left Join

DNI	Nombre	id_club
41555676	Juan Perez	1
38999432	Damian Lopez	4
33455654	Julio Suarez	1

Nombre_club	año_fundacion	id
River Plate	1901	1
Boca Juniors	1905	2

DNI	Nombre	id_club	Nombre_club	año_fundacion
41555676	Juan Perez	1	River Plate	1901
38999432	Damian Lopez	4	null	null
33455654	Julio Suarez	1	River Plate	1901

Right Join

DNI	Nombre	id_club
41555676	Juan Perez	1
38999432	Damian Lopez	4
33455654	Julio Suarez	1

Nombre_club	año_fundacion	id
River Plate	1901	1
Boca Juniors	1905	2

DNI	Nombre_apellido	id_club	Nombre_club	año_fundacion
41555676	Juan Perez	1	River Plate	1901
33455654	Julio Suarez	1	River Plate	1901
null	null	2	Boca Juniors	1905

Outer Join

DNI	Nombre	id_club
41555676	Juan Perez	1
38999432	Damian Lopez	4
33455654	Julio Suarez	1

Nombre_club	año_fundacion	id
River Plate	1901	1
Boca Juniors	1905	2

DNI	Nombre_apellido	id_club	Nombre	año_fundacion
41555676	Juan Perez	1	River Plate	1901
38999432	Damian Lopez	4	null	null
33455654	Julio Suarez	1	River Plate	1901
null	null	2	Boca Juniors	1905

OPERACIONES DE CONJUNTO CON TABLAS

Operaciones de conjunto

- Unión: Básicamente es un append. Es necesario que las 2 o mas tablas participantes tengan coumnas con el mismo tipo de datos.
- Excepción: Entre 2 tablas nos quedamos solo con lo que aparece en una pero en otra no.
- Intersección: uno solo se queda con lo que se repite en las tablas participantes.

Union

DNI	Nombre_apellido	id_club
41555676	Juan Perez	1
38999432	Damian Lopez	4
33455654	Julio Suarez	1

DNI	Nombre_apellido	id_club
41555676	Juan Perez	1
40444098	Tristan Katz	8

DNI	Nombre_apellido	id_club
41555676	Juan Perez	1
38999432	Damian Lopez	4
33455654	Julio Suarez	1
41555676	Juan Perez	1
40444098	Tristan Katz	8

Except

DNI	Nombre_apellido	id_club
41555676	Juan Perez	1
38999432	Damian Lopez	4
33455654	Julio Suarez	1

DNI	Nombre_apellido	id_club
41555676	Juan Perez	1
40444098	Tristan Katz	8

DNI	Nombre_apellido	id_club
38999432	Damian Lopez	4
33455654	Julio Suarez	1
40444098	Tristan Katz	8

Intersect

DNI	Nombre_apellido	id_club
41555676	Juan Perez	1
38999432	Damian Lopez	4
33455654	Julio Suarez	1

DNI	Nombre_apellido	id_club
41555676	Juan Perez	1
40444098	Tristan Katz	8

DNI	Nombre_apellido	id_club
41555676	Juan Perez	1
41555676	Juan Perez	1

PANDAS - UTILIZACIÓN BÁSICA

Qué es pandas?

- Pandas es una librería de Python especializada en manipulación y análisis de datos.
- Es una herramienta sumamente útil cuando queremos limpiar, analizar o simplemente manipular un dataset.
- Es canon en la industria del software y ampliamente utilizada

Instalación y manera de llamar

- No es necesario realizar un `pip install` ya que los paquetes vienen pre-instalados en las versiones actuales de python. Para importar la librería a tu script:
 - `import pandas as pd`
- Luego, el alias “pd” será usado para las funciones que corresponden a esta librería.
 - Ej: `df = pd.DataFrame({'Nombres': ['Juan', 'Gaston', 'Julian'], 'notas': [7,8,9]})`

Cómo leer archivos (.csv, .txt y .xlsx)

- Como dijimos, pandas es para manejar dframes, y ellos vienen en varios formatos
 - Leer un .txt o un .csv: `pd.read()`. Ejemplo:
 - `df_1= pd.read_csv('SMSSpamCollection.txt', sep="\t", header=None)`
 - `df_2=pd.read_csv('desagregado_por_dia.csv', sep="," , header=0)`
 - Leer un .xlsx:
 - `df_3=pd.read_excel('Historicalinvesttemp.xlsx', sheet_name='Solo_stock_positivo', header=0)`

Cómo leer archivos (.csv, .txt y .xlsx)

- Vemos que al leer textos planos (.csv y .txt) se debe ingresar el caracter por el cual las columnas están separadas (',';','-', etc) además del nombre del archivo a leer. Decir header=0 es decir que se use el primer registro a modo de header.
- Al leer .xlsx el campo de separador deja de ser necesario. Se agrega el campo opcional "sheet" ya que los excels admiten varias sheets. En caso de no pasar este parámetro, se lee la primera sheet.

Concepto de dataframe

- Todas las variables creadas en la filmina anterior dan como resultado dataframes.
- Los dataframes son objetos de la librería pandas, específicamente creados como representaciones de data tabular.
- Cada registro (fila) de un dataframe tiene un índice asociado
- A su vez, estos dataframes están compuestos de otras estructuras llamadas Pandas Series (columnas/atributos del df).

Concepto de dataframe

- Vimos cómo crear dataframes desde archivos.
- Crear un df vacío:
 - `df_vacío=pd.DataFrame()`
- Crear un df en particular: pasarle al comando un diccionario cuyas keys sean los títulos de las columnas y los vals sean listas con los datos.
 - `df = pd.DataFrame ({'Nombres': ['Juan','Gaston','Julian'], 'notas': [7,8,9]})`

Como indexar/manipular un df

- **df['campo_1']** devuelve los datos de la columna cuyo nombre es 'campo_1'. Si le asignáramos una variable, sería del tipo "series"
- **df['campo_1'][0]** devuelve el dato puntual correspondiente al atributo "campo_1" y al ÍNDICE 0. Fijarse que hablamos de índice, no de posición.
- **df.iloc[indice]** devuelve la fila correspondiente al índice en cuestión. Lo hace en forma de pd.series ("columna") solo que los índices pasan a ser los nombres de columna.
- **df.loc[indice,'columna_1']=valor** asigna un valor a determinada posición del df (a la que responda al índice y el nombre de columna)

Funciones útiles

- **len(df)** te dice cuantas filas hay en el df
- **list(df.columns)** arroja una lista donde cada elemento es el nombre de una de las columnas del df
- **df.fillna(valor)** reemplaza todos los nulls que pueda contener tu dataframe por un número que vos elijas
- **df.drop_duplicates()** elimina los duplicados dejando solo la primera aparición.

Funciones útiles

- **df.drop_duplicates(subset=[campo1,campo2,...], keep='first')**. Keep es por default first, puede ser 'last' quedándote con la última aparición. Puede ser False eliminando todo lo que estuvo duplicado alguna vez. Subset te dice los campos a considerar para la igualdad. Por default son todos.
- El comando **del df['campo']** elimina una columna
- **df.drop(indice)** elimina una row entera
- El drop puede dejar los índices desacomodados. Para volver a ordenar los índices de 0 a len(df)-1 se puede hacer **df.reset_index(drop=True)**. ¿Porque el drop?

Renombrar columnas, crear columnas nuevas

- Para renombrar se usa el método `rename` y se crea un diccionario donde la key es el nombre de la columna y el value su nombre nuevo.
 - `df.rename(columns={'oldName1': 'newName1', 'oldName2': 'newName2'}, inplace=True)`
- Agregar una columna “apendando una lista”. La lista debe tener la misma longitud que `len(df)`:
 - `df['campo_nuevo']=lista`
- Agregar una columna a partir de otra ya existente:
 - `df['campo_nuevo']=df['campo_cualquiera']*2`

Slicear un dataframe, quedarte con columnas/filas

- Slicear “verticalmente” (quedarte con determinadas columnas:
 - `df[['campo1','campo2','campo3']]`
- Slicear verticalmente (quedarte con un subset de filas). Esto es muy similar a slicear en listas:
 - `df[principio:final]`
- Slicear usando un condicional. Ej: Quedarte con un df cuyos registros tengan un stock mayor a 0.3. La condición se pone entre corchetes y se auto-referencia el df:
 - `df_stock=df_3[df_3['Stocks']>0.5]`

Shallow Copy vs DeepCopy

- Sucede algo similar a lo que sucedía con las lists y el **aliasing**. Dado un dataframe “df”, al decir `df1=df` estoy creando una referencia distinta al mismo objeto (el mismo set de datos). A esto se le llama ShallowCopy
- Al hacer `df.loc[indice,'campo_1']=valor_nuevo` ; tanto df como df1 sufrirán dicho cambio.
- Para crear realmente un set de datos duplicado (un objeto distinto) debemos realizar un **deepcopy**. Así:
 - `df1=df.copy(deep=True)`
- Notar que al hacer esto, duplicamos el set de datos, por lo que aumenta el uso de memoria de la instancia

Iterar sobre un dataframe

- Claramente, es muy tentador iterar sobre estas estructuras que acabamos de explicar. La manera correcta es mediante un for loop y el método “iterrows”.

```
#Iterando sobre un dataframe
def itera_df(df):
    columnas=list(df.columns)
    for index, fila in df.iterrows():
        print('este es el valor de la primera columna', fila[columnas[0]])
        print('este es el indice', index)
        print('esta es la fila', fila)
```

Iterar sobre un dataframe

- Qué se puede notar aca? Que para iterar sobre un dataframe hay que involucrar al ÍNDICE. Ya que se va iterando sobre el índice.
- Qué pasa si reordenamos/manipulamos el df de tal manera de que los índices se repiten, faltan, cambian de orden?
- La iteración falla.
- ¿Cómo podríamos resolver este potencial problema?

PANDAS - JOINS/OPERACIONES BOOLEANAS

Joins

- **Inner join:** `pd.merge(df1, df2, on="key")`. Siendo `df1` y `df2` los 2 dataframes a joinear y “key” el campo por medio del cual se va a realizar el join
- **Left Join:** `pd.merge(df1, df2, on="key", how="left")`
- **Right Join:** `pd.merge(df1, df2, on="key", how="right")`
- **Outer Join:** `pd.merge(df1, df2, on="key", how="outer")`

Union

- Dentro de las operaciones booleanas vistas en la última clase, podremos replicar la union mediante el “concat”. Dados 2 dataframes:
- `pd.concat([df1, df2])` realiza la unión entre ambos.

Ejercicios

- Realizar una función que efectúe una intersección entre 2 dataframes y otra que efectúe una excepción.
- Realizar un inner join, un outer join, un left join y un right join usando las tablas “customers” y “orders”, usando como clave de join ¿Qué atributo debiéramos usar para joinear ambas tablas?

Pandas en la labor de un D. Analyst

- Pandas es una herramienta excelente para LIMPIEZA Y MANIPULACIÓN de datasets.
- Puede “reemplazar” a SQL si no estás muy ducho
- Tiene ciertos módulos orientados a graficar datos, lo que ayuda a la hora de generar reportes.
- En combinación con numpy, se pueden generar datasets de entrenamiento para modelos de ML

Pandas en la labor de un D. Engineer

- Los dataframes son estructuras de datos con un considerable peso. Si se están manipulando volúmenes grandes (o el procesador de la VM donde corre el script es pequeño) puede resultar inconveniente.
- Si el job es más bien mediano/pequeño y se lo combina con drivers de SGBD (psycopg2, oraclegx, etc); es adecuado usarlo para jobs de ETL/ELT.